READING EDITION · ALL TOPICS

Compact Git knowledge

83 statements with explanations, 25 antipatterns with recommended practices and 24 terms. Fully available without playing or unlocking anything.

Myth or fact? The explanation.

M01 · MYTH

GitHub is public; GitLab is private.

Depending on the offer, both GitHub and GitLab support private and public repositories (i.e. managed project directories with version history). Visibility settings, user rights, server operation, external copies and the actual handling of data are crucial. "Private is an access setting, not a comprehensive data protection assessment.

M02 · MYTH

Git is the same as GitHub.

Git is the version management software; GitHub is a service that manages Git repositories and offers additional functions. The same applies to GitLab and other providers. Git also works without any hosting platform.

M03 · MYTH

Git is too complex for everyday editorial work.

Graphic user interfaces as well as integrations into work programs – such as Git support in the Oxygen XML Editor – simplify frequent tasks such as saving, comparing and matching changes with the common project status. Basic concepts and a careful handling of conflicts remain important; no one needs to know all commands for this.

M04 · MYTH

Git is only useful for teams.

Even earlier file versions can be restored, experiments can be carried out and the own processing process can be understood. This can be helpful for example with transcription and TEI markup.

M05 · MYTH

Git requires a server and internet access.

Git can work completely locally on a single computer. Only for synchronisation with other computers or people is an additional copy usually set up elsewhere. A commit is a deliberately stored version; for this no network connection is required.

M06 · MYTH

Git is a neutral tool with no influence on working culture.

The choice of processes, access options and evaluations influences which activities are documented, who is allowed to release changes and how contributions are perceived. An implicit expectation of work rhythm and accountability can arise from a tool.

M07 · MYTH

Deleting a file removes it from Git.

Previous saved version levels can still contain the file. An actual removal from the version history requires special measures and does not automatically consider copies from other parties.

M08 · MYTH

Publishing only the final edition automatically hides internal comments.

Those who make the repository accessible, including version history, can thereby disclose older texts, internal notes or temporarily contained files. The difference between the publication of a file export and the publication of the repository history is crucial.

M09 · MYTH

Using pseudonyms makes Git activity data anonymous.

People may be recognized by combination with other information. Pseudonymization and anonymisation are not synonymous; whether identity exists depends on the concrete context.

M10 · MYTH

Public Git data may be used for any employee analysis.

Commit metadata may be personal. An analysis must be evaluated, inter alia, with regard to the legal basis, purpose, necessity and protection of the data subject. In addition, such data are often methodologically inappropriate to measure working time or performance directly.

M11 · MYTH

Contributing to a public repository means consenting to every analysis.

Visibility is not a flat-rate consent. A processing can also be based on another legal basis under certain conditions, but this must be independently examined.

M12 · MYTH

An inference from Git activity is a fact about a person.

Git captures only certain documented actions and leaves many possible causes for a pattern open. Especially the linking of different data sources can create delicate, yet unreliable personal attributions.

M13 · MYTH

.gitignore safely prevents sensitive files from being published.

Already versioned files remain unaffected by this rule; previous versions are not cleaned afterwards. Also accidentally published access data require own countermeasures.

M14 · MYTH

Publishing a repository clears all reuse rights.

Text edition, image material, transcriptions, program code and third party contributions may affect different rights and licenses. Public readability alone does not grant a general re-use permit.

M15 · MYTH

Git records every editorial decision.

Conversations, undocumented alternatives, preparatory work and the motives behind a change need not be included in the repository. Versionation records results of certain work steps better than their mental development.

M16 · MYTH

git blame identifies the person intellectually responsible for a passage.

The last line intervention can be a formal correction, automatic reformatting or taking over of foreign work. The displayed assignment is technical and also depends on the examined version levels.

M17 · MYTH

Git saves every intermediate state automatically.

Usually only by a commit – an explicitly stored version – permanently comprehensible points in the project history are set. Unsaved experiments and discarded intermediates can be missing.

M18 · MYTH

Each commit represents exactly one editorial action.

A commit can bundle several changes or split a single work into many small entries. Granularity depends on habits and project rules.

M19 · MYTH

Commit order is the true chronological history of the work.

Work can be done long before it is saved. Functions such as Rebase (reordering or re-applying commits) or Squash (committing multiple commits) can be used to change the visible version sequence.

M20 · MYTH

Commit timestamps prove when editorial decisions happened.

Git distinguishes information about the original author's time and at the time of taking over into history (Author Date and Committer Date). Both are metadata and not automatically independently certified time data; the actual work may have been done earlier.

M21 · MYTH

The named commit author is certainly the actual contributor.

This information is generally configurable. In the case of jointly developed, imported or automated stored changes, the information may differ from the actual division of work. A separate signature or identity check is another question.

M22 · MYTH

Everything included in a commit is an intentional editorial statement.

In addition to technical changes, a commit can also include automatic formatting, sub-sequences of a script or accidentally saved adjustments.

M23 · MYTH

A large Git diff means a major editorial change.

Already a new indentation or reformatting of a XML file can make numerous lines appear as changed, although the editorial meaning remains almost the same.

M24 · MYTH

Reverting a change makes the earlier decision irrelevant.

The fact that a reading or markup was initially chosen and later rejected can be particularly revealing for the history of the edition. The technical undoing does not explain the reasons.

M25 · MYTH

Git prevents collaboration conflicts and data loss.

Conflicts can arise when merging two levels of development. They must be resolved properly. Unrecognized errors, incorrect operation or unsecured data are also possible.

M26 · MYTH

A conflict-free merge guarantees editorial consistency.

Two changes to different places can contradict each other in terms of content, although Git combines them technically easily – for example when markup guidelines are implemented differently in two places.

M27 · MYTH

Branches are automatically different scholarly versions.

Sectors are also used for technical repairs, experiments, work organization or short-term tests. Their scientific significance is derived from the concrete project practice.

M28 · MYTH

Git organizes teamwork by itself.

Teams need agreements on responsibilities, storage frequency, designations, conflict resolution, quality control and decision-making powers.

M29 · MYTH

A pull request is automatically scholarly peer review.

The examination can be limited to technical functionality or only take place formally. Content assessment requires its own criteria and responsibilities. The procedure is also a platform function, not an imperative feature of Git.

M30 · MYTH

Valid TEI XML is necessarily editorially correct.

Formal validity does not automatically verify the historical correctness of a reading, the justification of a normalization or the appropriateness of an markup.

M31 · MYTH

Text-based XML always yields readable Git comparisons.

Formatting changes, long XML lines or differently arranged elements make interpretation difficult. Git first compares characters or lines and does not understand the XML structure or editorial semantics by itself.

M32 · MYTH

A commit is a tamper-proof historical certificate.

checksum secure a technical relationship between data and characteristic values; they do not authenticate the truth of the registered author, time or decision information. Versions can be rebuilt, and the authenticity of a source depends on its transmission context.

M33 · MYTH

A Git snapshot makes an edition completely reproducible.

In addition, programs, external data, fonts, operating systems, configurations, databases and unversioned resources may be required. Versioned source data is an important building block, but not the entire execution environment.

M34 · MYTH

Git is a backup.

A single local copy does not protect against defects or loss of the device. Also synchronized copies can take over errors; unrecorded files and external storage may be missing. Backups require separate, tested recovery paths.

M35 · MYTH

A repository hosted on GitHub or GitLab is preserved for the long term.

Hosting contracts, accounts, storage rules, availability of external files and long-term development may change. Archiving requires organizational, technical and documentary measures beyond mere hosting.

M36 · MYTH

A DOI makes a Git repository immutable.

A DOI identifies a digital object or state of publication according to the rules of the respective service. Whether the associated data are actually kept as an unchangeable version depends on its procedures. An continuously changing repository and archived publication version must be distinguished.

M37 · MYTH

A tag guarantees a reliable published edition.

Tags technically mark a version, but do not justify a quality assurance or archiving process. Under certain conditions they can also be changed or deleted.

M38 · MYTH

Because Git is open source, projects are independent of platform vendors.

Additional platform functions – such as tickets for tasks, discussions, access management or automated processes – cannot always be transferred completely or loss-free to another service.

M39 · MYTH

A repository contains all project documentation.

Decisions from meetings, e-mails, external ticket systems and databases as well as unaccompanied work materials may be missing. Even a complete Git history is therefore not necessary a complete project transmission.

M40 · MYTH

Git works equally well for every media format.

Pure text files can often be compared particularly well. Large or binary files may require more memory and allow for less meaningful comparisons. Git LFS ("Large File Storage") is an extension in which large contents are stored separately and integrated into the Git repository via references; this creates its own requirements for backup and archiving.

M41 · MYTH

A Git history explains itself.

Edition guidelines, role distributions, used tools, work conventions and project phases only give changes their historical meaning. A conversion of the TEI markup can be based on technical, organizational or technical reasons.

M42 · MYTH

A Git diff automatically explains the editorial rationale.

The same visible change may have been caused by a new interpretation decision, an altered markup scheme, automatic conversion or correction of an error.

M43 · MYTH

More commits mean greater productivity.

People and projects store changes in different ways; automatic processes can generate additional entries. Number and quality of scientific decisions cannot be derived from the mere commit amount.

M44 · MYTH

No commits means no work happened.

Literature research, comparison of sources, discussion, proofreading, conceptual work and local editing without a subsequent commit can remain invisible.

M45 · MYTH

Commit timestamps show working hours.

Commits can be bundled, delayed, automated or created at times other than the actual work. They neither represent the entire activity nor necessarily its duration.

M46 · MYTH

Editing many files proves substantial editorial progress.

A single difficult source finding may require considerable effort, while automated transformations affect thousands of files. Technical progress requires project-specific criteria.

M47 · MYTH

Git statistics are directly comparable across edition projects.

Projects differ, among other things, by file size, tasks, number of employees, workflows, commit practice, degree of automation and repository type. A working repositorysitory usually documents ongoing processing unlike a publication repositorysitory, which contains mainly provided issues. Comparisons therefore need context and methodical normalization.

F01 · FACT

Oxygen XML Editor can control Git directly.

The Git Client Add-on for Oxygen XML Editor provides common Git functions in the usual XML working environment: Check differences, create commits, transfer and receive changes, manage development branches and handle conflicts. You don't have to type commands for these default tasks. If you work with TEI/XML, you can connect version management and text editing in one application, which lowers the entry threshold. The interface simplifies operation, but does not replace a basic understanding of version management. A conflict in merging must continue to be assessed in terms of content.

F02 · FACT

Uploading changes can trigger automatic validation.

A push (transfer of local commits to a remote repo) can trigger automated workflows on hosting platforms, often called CI/CD pipelines: defined test steps or processing steps that are executed after an event. TEI files can be automatically checked for well-formed XML, compliance with a Relax NG scheme or additional schematron rules. A faulty data level is noticeable early on. Git does not automatically perform a schema check through a push. To do this, you need installed programs and an execution environment such as GitHub Actions or GitLab CI/CD. schema validity is not a guarantee of editorial correctness.

F03 · FACT

A commit can trigger a new edition preview.

Automation can go beyond testing: With new commits using XSLT and other tools, you can create HTML pages, index, search indices or publication files from TEI/XML. The published view can be generated repeatedly and easily from a specified data status; changes become visible in a timely manner. This is an additional publication process, not a built-in property of Git. Software versions, external data and settings must also be documented for repeatability.

F04 · FACT

Repositories can require checks before accepting changes.

Hosting platforms support protection rules for important industries (development branches). One can, for example, demand that automatic tests are successful or a second person has checked the change before entering the common main version. A team can integrate minimum technical standards – such as successful XML validation – into the workflow. Such rules apply only where they can be set up and not circumvented. A approval is not automatically a scientific review.

F05 · FACT

Programs can run even before a commit is saved.

Git Hooks are scripts that respond to certain Git events. For example, a pre-commit hook is executed before creating a new commit. On a local machine, a XML check, format check or search for accidentally recorded files can start automatically. Local hooks need to be set up and can be bypassed. They do not replace central rules for jointly published data.

F06 · FACT

Several project versions can be open simultaneously.

With Git Worktree you can create several work directories that show on different branches of the same repository. One can look at the published edition in one directory and test a new TEI markup strategy in parallel in another. The separation concerns working directories, not automatically independent data stocks, access controls or archive copies.

F07 · FACT

A commit holds much more than a message.

A commit contains a unique identifier, a reference to the stored file stock, references to previous commits, information about author and creator, time details and a message. If necessary, a digital signature is added. The differences between two levels can be calculated from it and from the corresponding file stocks. Technical sequences, attributions and editing contexts can be investigated – even if the explaining message is scarce. The metadata does not easily prove the identity of the person acting, the length of work or the reasons for a decision.

F08 · FACT

A commit can have a different author and committer.

Git distinguishes author (who is attributed the change) and committer (who has created the commit in its present form). This helps to understand more differentiatedly submitted contributions, adopted changes or the transfer of older project portfolios. Both data are configurable metadata, not automatically verified proofs of identity.

F09 · FACT

You can trace the recorded history of individual lines.

git blame can show which commit has last changed a line; certain options also take into account moved or copied lines. The creation of a read or TEI markup can be traced back to an earlier editing state. When reformatting, the assignment can be misleading. The last documented intervention does not automatically say anything about scientific responsibility.

F10 · FACT

You can search history for a particular text change.

Git can search for commits where a string has been added or removed. The known search function is Pickaxe. For example, it is possible to examine when certain TEI elements, name forms or references became more common or rarer. The search is string-oriented and not a full-fledged semantic XML comparison. It does not automatically reveal any relevant change in content.

F11 · FACT

You can compare project states years apart.

A diff is the representation of differences between two editing levels. Git can also compare states between which years and very many commits lie. Long-term restructurings, new document groups and the development of an edition will become visible. The comparison shows differences between the chosen states, not all intervening considerations or decisions.

F12 · FACT

Git can infer that a file was renamed.

When comparing editing positions, Git can use similar content to suggest that a file has been moved or renamed. This helps track texts by changing folder structure, signatures or file names. A renaming is not generally stored in Git as an own event; the recognition depends on comparison options and similarities.

F13 · FACT

Git can locate the first version containing a fault.

git bisect gradually checks older commits to limit the transition from a functioning to a faulty state. Helpful if an XSLT transformation, a index or the generation of an edition website has eventually become flawed. One needs a useful test, with which it can be determined for examined versions whether the problem already occurs.

F14 · FACT

You can annotate an older commit without rewriting it.

Git Notes are additional notes that can be assigned to a commit without changing its original content. A later source-critical classification can remain separate from the originally handed down editing. Such notes are not displayed in all programs and are not automatically transmitted with each synchronization.

F15 · FACT

Some apparently lost commits can be recovered.

The local reflog records where Git-Vweise last showed. This may also be possible after an accidental reset of older commits. Useful in case of operating errors or accidentally discarded developmental branches. The records are local, can run and be cleaned up. Reflog is not a backup.

F16 · FACT

Project history can be viewed as a branching diagram.

Graphical Git programs represent commits as points and their technical relationships as connecting lines. Thus one sees development branches and their merging (merge) at a glance. Parallel processing, integration phases and longer development lines become easier to understand. The diagram shows relationships between stored levels, not all communication or decision-making processes.

F17 · FACT

Git history can become an animated landscape.

The Gource visualization program shows the development of a repository as an animation: directories, files and logged posts appear as moving elements. Very suitable to show the emergence, growth and restructuring of a project in a lecture. The animation is based on the visible Git data. It does not show the entire workload and can make patterns appear aesthetically more impressive than they are methodically resilient.

F18 · FACT

Git metadata can power activity charts.

Commit times, modified files and documented editing frequencies can be statistically processed and visualized. This allows the study of work phases, technical restructuring and different project practices. The number of commits is neither a reliable proof of working time nor an objective measure of productivity. Personal analyses raise additional data protection questions.

F19 · FACT

Git can highlight differences within a line.

Git can also highlight smaller text elements as differences instead of completely changed lines; this representation is called Word Diff. Especially helpful for long XML lines where only one word or a small section has been changed. A word comparison is not yet a comparison of the semantic meaning of TEI elements.

F20 · FACT

Git comparisons can be adapted to file types.

With a Git configuration file called .gitattributes, file types can be handled differently and own comparison programs can be integrated. XML files can be prepared specifically for comparisons in order to better separate purely technical formatting changes from content-related interesting interventions. A project-specific comparison requires additional institution and is not automatically scientifically interpretative.

F21 · FACT

Git works fully offline.

Versioning files, creating commits, comparing differences and managing industries works without hosting platform or Internet access. Individuals can also make their own transcription and markup practice comprehensible. Without an additional, independently stored copy, the local version history is not protected from device loss.

F22 · FACT

A Git history can be packed into one portable file.

A Git Bundle is a portable file containing selected Git objects and references of a repository. It can be passed on and reread without a running Git server. Suitable for offline transfers, technical project transfers and certain backup scenarios. A bundle does not automatically contain unsaved working files, all local settings or externally stored content such as Git LFS objects.

F23 · FACT

You can export just the files from a historic version.

With git archive, you can export the files of a particular commit as ZIP or TAR archives – without giving the whole Git history. This enables a clearly defined publication status to be published, while the internal work history remains in the project. A file export without a version history does not document how the data was created. External dependencies must also be taken into account separately.

F24 · FACT

Published states can be given memorable names.

A tag is a named tag of a particular commit – similar to a bookmark in the history of the project. edition-1.0 could, for example, refer to the version of a published edition. Some versions are found and can be cited or exported more easily. A tag is not yet a publication, no DOI and no archival proof. Such markings can be changed or deleted technically.

F25 · FACT

Commits and tags can carry digital signatures.

Certain Git objects can be signed cryptographically. Under suitable conditions, it is possible to check whether the signed version matches the signature used and the corresponding key. Signatures may support technical evidence of a released version. They prove neither the contentual correctness of the edition nor the real identity or scientific responsibility of a person alone.

F26 · FACT

You can link to lines in an exact historical version.

Platforms such as GitHub can provide more permanent created links to files in a specific commit, if necessary up to the single line. In a research discussion you can refer to the exact version of a TEI file at that time rather than to the always variable current version. Permanent accessibility depends on storage and accessibility. Line numbers are not editorially stable text anchors.

F27 · FACT

Text and editing tools can have a shared history.

In addition to TEI files, schema files, XSLT transformations, scripts, ODD specifications and configurations can be versioned in the same repository. A later comparison can show whether changes have been made to the data display along with changes to the edition guidelines or technical processing. The joint storage does not yet generate an automatic explanation of causal relationships. External systems must be additionally documented.

F28 · FACT

You can experiment with editorial rules without replacing the main version.

A branch is a branch of development in which changes can be continued independently of the shared main status. Later, the work can be reunited (merge) if necessary. Alternative TEI markup models, normalizations or transformation paths can be tested and compared. The later merging can create technical conflicts. Also conflict-free changes can be editorially incompatible.

F29 · FACT

A commit can contain only selected changes.

Git can select individual change sections for a commit. This is often called partial staging – i.e. the targeted prefixing of only part of the changes. An orthographic correction and a modification of the XML markup can be documented separately, although they concern the same file. Selected substeps can result in temporarily invalid XML. Therefore, the overall state to be saved should be checked.

F30 · FACT

Unfinished work can be temporarily put aside.

With Git Stash, changes not yet recorded in a commit can be temporarily set aside and replayed later. One can make an urgent correction in the short term and then return to a longer, still unfinished processing. This intermediate storage is not a long-term scientific documentation and no secure backup strategy.

F31 · FACT

Cloning a repository often copies its earlier history too.

A clone is a local copy of a repository. Normally not only the current files, but also the achievable Git history are transferred. A repo can be examined and processed independently of the server connection. There are also limited copying procedures. For data protection and publication, it is crucial that historical metadata and previous file versions can be shared.

F32 · FACT

Different author names can be combined for display and statistics.

With a file called .mailmap, different names or email addresses of the same person can be unified for certain ads and evaluations without rewriting the old commits. Particularly important in long-term projects where stakeholders appear under several addresses or spellings. The assignment is interpretative and must be justified; the same names do not necessarily mean equal persons.

F33 · FACT

Files can deliberately be compared differently.

About .gitattributes, for example, Git can dispense with the usual line-by-line comparison display for certain files or use external comparison tools. For created, binary or structurally special files you can choose a suitable representation. A different presentation does not hide the content from legitimate users; it is not a data protection or access protection function.

F34 · FACT

Large media can live outside ordinary Git objects.

Git Large File Storage (Git LFS) adds a procedure for large files: There are references in the Git repository, the actual content is stored in an additional storage structure. Helpful for high resolution facsimiles, audio/video files or large binary stocks. For complete handovers and long-term preservation the external file objects must be explicitly backed up.

F35 · FACT

Working history and published data can be kept separate.

You can run a comprehensive working repositorysitory internally and only provide audited files or an adjusted data export to the public – also automated. This allows transparency over published research data without having to disclose all internal discussion and work metadata. The separation has to be set up conceptually and technically reliably; a mere visibility setting does not clean up an already published version history.

F36 · FACT

A Git project is portable between hosting platforms.

The actual Git history can basically be transferred to another service or institutional server. This creates scope for migration, institutional control and long-term availability of research data. Platform-based additional information such as tickets (issues), discussion processes, approvals or access rules are not automatically part of the portable Git history.

Recognise antipatterns and improve practice

A seemingly plausible solution can create disadvantages in particular conditions. The boundary explains when the practice may still be appropriate.

A1

The version in the filename

Pattern: Save every version as a separate file with version numbers, dates and names in the filename.

Better: Use version control with stable filenames; identify defined exports with clear version identifiers.

Boundary: Version identifiers in immutable publication exports are legitimate. Using file copies as the sole means of working version control is problematic.

A2

The copy zoo

Pattern: Exchange independently edited folder copies informally across computers, drives and storage devices.

Better: Use coordinated synchronisation with version control and independent backups.

Boundary: Controlled backups, archival snapshots and explicitly immutable exports are useful.

A3

History without a past

Pattern: Overwrite earlier working versions or discard them after publication.

Better: Use version control and a deliberate preservation strategy, reviewed for legal requirements, for scientifically relevant process documentation.

Boundary: Not every transient draft warrants archiving. A publication repository need not contain the entire internal working history.

D3

Git is our backup

Pattern: Let the existing Git repository replace all independent backup procedures.

Better: Document and test a backup and recovery strategy in which Git can be one component.

Boundary: Git can form a substantial part of a robust backup strategy.

D4

The ownerless repository

Pattern: Leave administration, rights clearance and decisions informally with individuals, without substitution or handover arrangements.

Better: Document institutional, scholarly, technical and data-protection responsibilities, including deputies and handover procedures.

Boundary: Individuals may be responsible, but responsibility must be transferable and appropriately secured.

B1

Metadata on the move

Pattern: Keep them exclusively in commit messages, tags, branches, issues, filenames or folders instead of suitable structured metadata.

Better: Use TEI headers where appropriate, such as revisionDesc, scholarly metadata files or another suitable structured model; add Git references as a supplement.

Boundary: Provisional or redundant Git information is legitimate. The problem arises when necessary long-term scholarly information remains exclusively there.

B2

The branch as an edition model

Pattern: Represent their scholarly identity and relationships exclusively through branches.

Better: Model scholarly representations and relationships explicitly, using transformations where appropriate; develop in branches and use defined releases for citable versions.

Boundary: Even long-lived version branches can be appropriate if their scholarly meaning is explicitly documented.

B3

The hidden editorial decision

Pattern: Keep the only justification in commit messages, issues or pull-request discussions.

Better: Move publication-relevant justifications into the apparatus, commentary or edition documentation; link Git discussions as supplementary material.

Boundary: Not every working discussion needs publication; the necessary justification of the published edition matters.

B4

The orphaned editorial status

Pattern: Automatically treat an operational status such as a closed issue, merged branch or done column as scholarly approval.

Better: Define an explicit, traceable approval process with stated checks and a documented result.

Boundary: A project board can represent approval if the status is tied to verifiable criteria and responsibilities.

C5

The unchecked publication

Pattern: Publish new working versions directly without defined quality checks.

Better: Use validation, project-specific tests and explicit publication approval.

Boundary: Public experimental working versions may be unfinished if clearly labelled. Direct commits to main are not the problem in themselves.

G1

The unspoken Git agreement

Pattern: Treat repository access as implicit agreement on rights, duties and future uses of working data.

Better: Provide binding, understandable repository rules, supplemented by legally required agreements and information.

Boundary: Existing institutional rules may suffice; not every issue requires a separate individual contract.

D1

Access forever

Pattern: Grant permissions on entry but never review or revoke them when tasks change or people leave.

Better: Assign only role-appropriate necessary rights, review them regularly and define onboarding and offboarding, including service accounts.

Boundary: Ongoing work can justify explicitly renewed permissions. Revocation does not prevent use of copies already made.

G4

The unclear chain of rights

Pattern: Treat the ability to commit contributions as sufficient permission for all later uses.

Better: Clarify and document protection status, rights holders and required permissions for different contribution types early.

Boundary: Not every contribution is protected by copyright; contracts or other rules may already settle rights. Applicable law matters.

G5

Licensing at the click of a button

Pattern: Apply a single licence indiscriminately to TEI, scholarly texts, software, translations and third-party material.

Better: Provide clear resource-specific rights and licensing information; use a single licence only where rights are genuinely uniform.

Boundary: A uniform licence is useful if it effectively covers every relevant resource.

G2

Blanket consent

Pattern: Treat a broad initial declaration as permission for all current and future analyses and publications.

Better: Clarify purposes, suitable legal bases, information and safeguards for each use; obtain valid specific consent where appropriate.

Boundary: Voluntary, informed and sufficiently specific consent can be legitimate in suitable circumstances; it is not the only possible legal basis.

E1

The commit as proof of performance

Pattern: Interpret commit counts, changed lines and frequency directly as performance or productivity.

Better: Use scholarly, contextual criteria and qualitative assessment, including work not recorded in Git.

Boundary: These numbers can be useful as descriptive project metrics. Permission for processing personal data is a separate question addressed by G3.

E2

Git as an all-knowing witness

Pattern: Treat Git history as a complete record of scholarly work and decision-making.

Better: Use Git as a limited source alongside other process sources and contextual knowledge.

Boundary: Claims about Git activity itself are possible when their scope is stated accurately.

G3

Purpose drift in Git data

Pattern: Reuse personal version-control data for attendance monitoring, individual performance assessment or personnel decisions without a separate assessment.

Better: Assess each analysis purpose and legal basis; use suitable access restrictions, data minimisation and aggregated analysis where appropriate.

Boundary: Project-related analyses can be legitimate. This is separate from whether the resulting metric is methodologically sound, addressed by E1.

G6

The unlimited working chronicle

Pattern: Keep complete personal Git histories indefinitely without defined purposes, retention periods, access rules or safeguards.

Better: Define a justified retention and archiving policy with differentiated access, safeguards and consideration of scholarly and personal interests.

Boundary: Long-term scholarly archiving can be lawful and useful but needs suitable legal and organisational conditions.

D2

The forgotten Git past

Pattern: Clean only the current data while publishing the earlier Git history unchanged.

Better: Review the entire dataset to be released, including history; consider a separate publication repository.

Boundary: Permissible internal preservation may be useful. Rewriting history alone does not reliably remove copies already distributed.

C1

The tangled commit

Pattern: Bundle unrelated changes into one commit without a recognisable common unit.

Better: Group logically related changes; document separate bulk operations.

Boundary: A large coherent data import is legitimate and particularly common in publication repositories.

C2

The uninformative history

Pattern: Use messages that give insufficient indication of a commit's subject or purpose.

Better: Write brief, informative messages following an understandable convention.

Boundary: Short or automatically generated messages are not inherently problematic if their purpose is clear enough.

C3

Integration postponed

Pattern: Keep shared changes in separate branches for a long time and reconcile them only late.

Better: Reconcile and integrate regularly according to the actual dependencies between work packages.

Boundary: Long-lived separate branches are legitimate if tasks are independent or separation is deliberately justified.

C4

The formatting tsunami

Pattern: Let editors reformat indentation, whitespace or line breaks extensively during small scholarly edits.

Better: Use stable shared editor and formatting conventions; commit formatting-only changes separately.

Boundary: Deliberate formatting migrations are legitimate; detailed line history may matter less in a publication repository.

C6

The moving publication

Pattern: Identify only the moving tip of a branch such as main as the published version.

Better: Provide immutable, clearly identified releases and persistent identifiers where suitable, alongside latest access.

Boundary: An always-current version is useful if earlier publication versions remain unambiguously referenceable.

Terms

Git

Software for recording, managing and comparing saved versions of files.

Repository / Repo

A project folder managed by Git, together with its recorded version history.

Commit

A recorded snapshot of a project, with metadata and a message.

Commit message

A short description attached to a saved commit.

Push

Transfer local commits to another repository, often on a server.

Pull

Fetch and integrate changes from a remote repository.

Clone

Create a local copy of a repository, usually including its reachable history.

Branch

A movable pointer marking a line of development, for example an experiment.

Merge

Combine development histories, sometimes requiring conflicts to be resolved.

Tag

A name attached to a particular commit, similar to a bookmark.

Diff

A comparison showing differences between two saved states.

Staging / Partial staging

Choosing which changes will go into the next commit; individual parts of a file can be selected.

Hook

A program Git runs at a particular event, such as before recording a commit.

CI/CD

Automated processes for checking, building and publishing project changes.

GitHub / GitLab

Hosting platforms with collaboration tools built around Git; they are not Git itself.

Git Notes

Additional annotations attached to commits without rewriting the commits.

Reflog

A local record of where Git references pointed recently; sometimes useful for recovery.

Git LFS

Git Large File Storage: an extension for managing large files outside ordinary Git objects.

.gitattributes

A configuration file that controls how Git handles selected file types.

.mailmap

A configuration file used to normalize author names and email addresses for display.

TEI/XML

Structured text encoded using XML following the Text Encoding Initiative guidelines.

Relax NG / Schematron

Technologies for validating XML structure and more specific rules.

XSLT

A language for transforming XML, for example into HTML.

ODD

The TEI specification format used to describe and customize a project's encoding rules.

Sources: Git Discovery (83 statements, 24 terms) and Git Antipattern Academy (25 cards). All topics from both collections are included; relationship graphs and extended game scenarios are outside this compact reading edition.